

# PopOut, Monte Carlo Tree Search e Decision Trees

Bruno Souza Zabet - 202302069

Joe

Vitor

Este notebook é o walkthrough do projeto de Inteligencia Artificial 2025/2026:

## Adversarial Search Strategies and Decision Trees.

O objetivo é mostrar, de forma reprodutível, como o projeto cumpre o enunciado:

- implementar o jogo **PopOut**, uma variante de Connect-4 com movimentos **drop** e **pop** ;
- implementar uma estratégia adversarial com **Monte Carlo Tree Search (MCTS)** usando UCT;
- explorar melhorias ao MCTS base, incluindo rollouts táticos, vies por coluna central, bloqueios e paralelização;
- implementar uma **Decision Tree ID3** própria, sem usar scikit-learn para treinar a árvore;
- discretizar dados numéricos no warm-up Iris;
- gerar um dataset PopOut (**state, best\_move**) com MCTS;
- treinar uma Decision Tree para imitar as escolhas do MCTS;
- demonstrar os modos pedidos: humano vs humano, humano vs computador e computador vs computador.

Restrições importantes consideradas:

- a Decision Tree é implementada no projeto em **src/decision\_tree**, sem treino automático por bibliotecas externas;
- MCTS joga sobre clones do estado para não alterar o jogo real durante a pesquisa;
- PopOut tem regras adicionais para vitória simultânea, empate por tabuleiro cheio e repetição de estado;
- a árvore PopOut aprende a política do MCTS, ou seja, o alvo do dataset e o movimento recomendado pelo algoritmo de procura.

## 1. Setup

Importa as bibliotecas e módulos principais

```
In [11]: import ast
import random
from pathlib import Path
from time import perf_counter
```

```

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import SVG, display

from src.decision_tree.id3_decision_tree import ID3DecisionTree
from src.decision_tree.metrics import (
    conditional_entropy,
    entropy,
    information_gain,
)
from src.discretizer import apply_bins, learn_bins
from src.mcts.mcts_service import MCTS
from src.mcts.mcts_service_parallel import ParallelMCTS
from src.pop_out import (
    BLUE_PLAYER,
    COLS,
    DRAW_MOVE,
    EMPTY,
    RED_PLAYER,
    ROWS,
    PopOut,
)
from src.popout_decision_tree.data_preparation.common import (
    FULL_TRAINING_DATASET,
    merge_datasets,
)
from src.popout_decision_tree.train_popout_dt import (
    DOT_PATH,
    MODEL_PATH,
    load_decision_tree,
    train_popout_decision_tree,
)

PROJECT_ROOT = Path.cwd()
DATASET_PATH = Path(FULL_TRAINING_DATASET)
TREE_SVG_PATH = PROJECT_ROOT / "src" / "popout_decision_tree" / "datasets"

pd.set_option("display.max_columns", 80)
pd.set_option("display.width", 120)

print(f"Project root: {PROJECT_ROOT}")
print(f"PopOut dataset: {DATASET_PATH}")
print(f"Trained model: {MODEL_PATH}")

```

Project root: /home/bruno-zabot/Desktop/assignments/artificial-intelligence

PopOut dataset: /home/bruno-zabot/Desktop/assignments/artificial-intelligence/src/popout\_decision\_tree/datasets/popout\_training\_full.csv

Trained model: /home/bruno-zabot/Desktop/assignments/artificial-intelligence/src/popout\_decision\_tree/datasets/popout\_DT\_model.pkl

## 2. Mapa do código

Antes de discutirmos os algoritmos, esta secção lista as classes e funções implementadas. Isto ajuda na apresentação, porque cada parte do projeto fica ligada a um ficheiro concreto.

## Resumo dos modulos principais

| Ficheiro                                                    | Papel no projeto                                                                     |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>src/pop_out.py</code>                                 | Regras do PopOut, validacao de movimentos, vitorias, empates e loop humano vs humano |
| <code>src/mcts/mcts_service.py</code>                       | MCTS generico com selecao UCT, expansao, simulacao e retropropagacao                 |
| <code>src/mcts/mcts_service_parallel.py</code>              | Variante paralela que agrega pesquisas independentes na raiz                         |
| <code>src/mcts/pop_out_mcts.py</code>                       | Adaptador de jogo PopOut contra MCTS no terminal                                     |
| <code>src/decision_tree/*.py</code>                         | Implementacao propria do ID3, nos da arvore e metricas de entropia/ganho             |
| <code>src/popout_decision_tree/data_preparation/*.py</code> | Geracao e fusao de datasets PopOut rotulados por MCTS                                |
| <code>src/popout_decision_tree/train_popout_dt.py</code>    | Treino, exportacao e carregamento da Decision Tree PopOut                            |
| <code>src/play_popout.py</code>                             | Interface final com humano, random, MCTS e Decision Tree                             |

## 3. PopOut

O jogo usa um tabuleiro 6x7. Os valores internos sao:

- `0` : célula vazia;
- `1` : jogador vermelho;
- `-1` : jogador azul.

Cada movimento legal e representado como `(coluna, acao)` :

- `(4, "d")` : drop de uma peça na coluna 4;
- `(4, "p")` : pop de uma peça propria do fundo da coluna 4;
- `"draw"` : declarar empate quando a regra permite.

A classe `PopOut` também implementa a regra de vitória simultanea: se um movimento criar quatro-em-linha para ambos os jogadores, vence quem acabou de jogar.

```
In [18]: # Exemplo pequeno e deterministico de movimentos drop e pop.
game = PopOut()
game.turn = RED_PLAYER
```

```

scripted_moves = [(4, "d"), (5, "d"), (4, "d")]
for move in scripted_moves:
    assert game.is_valid(move), f"Movimento invalido no exemplo: {move}"
    game.apply_move(move)

print("Turno atual:", "RED" if game.turn == RED_PLAYER else "BLUE")
print("Movimentos legais:", game.available_moves())
print("Vencedor:", game.check_winner())
game.print_board(show_arrow=False)

```

Turno atual: BLUE

Movimentos legais: [(1, 'd'), (2, 'd'), (3, 'd'), (4, 'd'), (5, 'd'), (5, 'p'), (6, 'd'), (7, 'd')]

Vencedor: None

```

|●●●●●●●●|
|●●●●●●●●|
|●●●●●●●●|
|●●●●●●●●|
|●●●●●●●●|
|●●●●●●●●|
|●●●●●●●●|

```

Funcoes centrais do motor **Pop0ut** :

| Funcao                         | Descricao breve                                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>available_moves()</code> | Devolve todos os movimentos legais para o jogador atual                                                            |
| <code>is_valid(move)</code>    | Valida <code>drop</code> , <code>pop</code> ou <code>draw</code> no estado atual                                   |
| <code>apply_move(move)</code>  | Aplica o movimento, atualiza <code>last_player</code> e troca o turno                                              |
| <code>clone()</code>           | Cria uma copia independente usada pelo MCTS                                                                        |
| <code>check_winner()</code>    | Procura vitorias horizontais, verticais e diagonais, incluindo vitoria simultanea                                  |
| <code>is_terminal()</code>     | Diz se o jogo terminou por vitoria, empate ou ausencia de movimentos                                               |
| <code>get_board_state()</code> | Representa o estado como tuplo imutavel ( <code>turn</code> , <code>flat_board</code> ) para datasets e repeticoes |

## Como o empate (draw) foi implementado no PopOut

O enunciado define duas situacoes especiais de empate: tabuleiro cheio e repeticao do mesmo estado tres vezes. A implementacao trata essas regras no motor do jogo e tambem no runner de partidas.

| Regra do enunciado                                        | Implementacao                                                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Tabuleiro cheio                                           | <code>PopOut.is_full_board()</code> verifica se todas as celulas estao ocupadas                                 |
| Jogador pode declarar empate se o tabuleiro estiver cheio | <code>available_moves()</code> adiciona <code>DRAW_MOVE</code> quando <code>is_full_board()</code> e verdadeiro |
| Movimento <code>draw</code> so e legal quando             | <code>is_valid(DRAW_MOVE)</code> devolve <code>True</code> apenas com tabuleiro cheio                           |

| Regra do enunciado             | Implementacao                                                                                                                          |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| permitido                      |                                                                                                                                        |
| Aplicar empate                 | <code>apply_move(DRAW_MOVE)</code> define <code>game_drawn = True</code> e nao troca para uma jogada normal                            |
| Fim de jogo por empate         | <code>is_terminal()</code> devolve <code>True</code> quando <code>game_drawn</code> e verdadeiro                                       |
| Vencedor em empate             | <code>check_winner()</code> devolve <code>None</code> quando <code>game_drawn</code> e verdadeiro                                      |
| Repeticao de estado tres vezes | <code>PopOut.run()</code> e <code>PopOutMatch.handle_repeated_state_draw()</code> contam <code>get_board_state()</code> num dicionario |

Esta decisao e importante para o MCTS: como `DRAW_MOVE` aparece em `available_moves()`, o algoritmo consegue considerar o empate como uma acao legal quando o proprio estado do jogo permite. Durante rollouts, `MCTS._is_repeated_draw()` tambem detecta repeticoes usando `get_board_state()` e trata o resultado como empate.

## Todas as funcoes da classe `PopOut`

Esta tabela resume todos os metodos definidos em `src/pop_out.py`.

| Funcao                                    | Descricao                                                                                                                                         |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__()</code>                   | Cria um tabuleiro vazio, escolhe aleatoriamente o jogador inicial e inicializa o estado interno do jogo.                                          |
| <code>print_turn()</code>                 | Mostra no terminal qual jogador tem a vez atual, usando cores diferentes para vermelho e azul.                                                    |
| <code>print_board(show_arrow=True)</code> | Imprime o tabuleiro no terminal e, opcionalmente, mostra setas para indicar colunas onde e possivel fazer <code>drop</code> ou <code>pop</code> . |
| <code>get_board_state()</code>            | Devolve uma representacao imutavel e plana do estado, incluindo o jogador atual; e usada para repeticoes, datasets e MCTS.                        |
| <code>clone()</code>                      | Cria uma copia independente do jogo para que o MCTS possa simular movimentos sem alterar o tabuleiro real.                                        |
| <code>available_moves()</code>            | Devolve todos os movimentos legais do jogador atual, incluindo <code>draw</code> quando o tabuleiro esta cheio.                                   |
| <code>apply_move(move)</code>             | Aplica um movimento, atualiza <code>last_player</code> , troca o turno e devolve o jogador que acabou de jogar.                                   |
| <code>next_player(player)</code>          | Devolve o adversario do jogador recebido, trocando <code>1</code> por <code>-1</code> e vice-versa.                                               |

| Funcao                                                     | Descricao                                                                                                                                                                    |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>is_terminal()</code>                                 | Indica se o jogo terminou por empate, vitoria ou ausencia de movimentos legais.                                                                                              |
| <code>get_ans()</code>                                     | Le uma resposta <code>yes/no</code> do terminal e devolve apenas <code>y</code> ou <code>n</code> ; e usada na regra de empate por repeticao.                                |
| <code>is_full_board()</code>                               | Verifica se todas as casas do tabuleiro estao ocupadas.                                                                                                                      |
| <code>print_invalid_move(is_board_full)</code>             | Mostra a mensagem de erro correta quando o jogador introduz um movimento invalido.                                                                                           |
| <code>get_player_input(is_board_full)</code>               | Le, interpreta e valida a entrada do jogador humano ate obter um movimento legal ou <code>draw</code> .                                                                      |
| <code>is_valid(move)</code>                                | Valida se um movimento e legal no estado atual: <code>drop</code> exige topo vazio, <code>pop</code> exige peca propria no fundo, e <code>draw</code> exige tabuleiro cheio. |
| <code>make_move(move)</code>                               | Executa fisicamente um <code>drop</code> ou <code>pop</code> no tabuleiro, deslocando as pecas quando necessario.                                                            |
| <code>is_winning_line(row, col, row_step, col_step)</code> | Verifica se existem quatro pecas iguais a partir de uma posicao numa direcao especifica.                                                                                     |
| <code>check_horizontal_wins()</code>                       | Procura sequencias vencedoras horizontais e devolve os jogadores que venceram nessa direcao.                                                                                 |
| <code>check_vertical_wins()</code>                         | Procura sequencias vencedoras verticais e devolve os jogadores que venceram nessa direcao.                                                                                   |
| <code>check_diagonal_wins_backslash()</code>               | Procura sequencias vencedoras na diagonal <code>\</code> .                                                                                                                   |
| <code>check_diagonal_wins_slash()</code>                   | Procura sequencias vencedoras na diagonal <code>/</code> .                                                                                                                   |
| <code>check_winner()</code>                                | Junta os resultados das quatro direcoes e devolve o vencedor; se ambos vencerem, ganha quem fez o ultimo movimento.                                                          |
| <code>print_winner(player)</code>                          | Imprime no terminal a mensagem de vitoria do jogador recebido.                                                                                                               |
| <code>run()</code>                                         | Executa o loop interativo completo: imprime turno/tabuleiro, trata repeticoes, recebe jogadas, aplica movimentos e termina em vitoria ou empate.                             |

```
In [ ]: # Demonstracao simples da regra de draw por tabuleiro cheio.
draw_game = PopOut()
draw_game.turn = RED_PLAYER

# Preenche o tabuleiro sem criar uma situacao real de partida; o objetivo
```

```
# e isolar a regra de disponibilidade do movimento DRAW_MOVE.
draw_game.board = [
    [RED_PLAYER if (row + col) % 2 == 0 else BLUE_PLAYER for col in range(
        COLUMNS)] for row in range(ROWS)]

# print tabuleiro ---

print("Tabuleiro cheio:", draw_game.is_full_board())
print("DRAW_MOVE esta nos movimentos legais:", DRAW_MOVE in draw_game.available_moves())
print("DRAW_MOVE e valido:", draw_game.is_valid(DRAW_MOVE))

draw_game.apply_move(DRAW_MOVE)
print("Jogo terminou:", draw_game.is_terminal())
print("Vencedor:", draw_game.check_winner())
print("Flag game_drawn:", draw_game.game_drawn)
```

```
Tabuleiro cheio: True
DRAW_MOVE esta nos movimentos legais: True
DRAW_MOVE e valido: True
Jogo terminou: True
Vencedor: None
Flag game_drawn: True
```

## 4. Monte Carlo Tree Search

A implementacao em `src/mcts/mcts_service.py` e generica: qualquer jogo pode ser usado se implementar o contrato `clone`, `available_moves`, `apply_move`, `is_terminal` e `check_winner`.

Cada iteracao do MCTS segue quatro fases:

1. **Selection:** desce na arvore escolhendo o filho com maior UCT.
2. **Expansion:** cria um novo filho para um movimento ainda nao explorado. Neste projeto, o filho recebe tambem um `prior_score` heuristico, calculado a partir do estado pai e do movimento escolhido. (explicamos melhor abaixo)
3. **Simulation:** joga uma partida simulada a partir desse estado. As primeiras jogadas do rollout usam regras taticas simples (explicamos melhor abaixo); depois o rollout volta a uma escolha mais leve baseada em preferencia central.
4. **Backpropagation:** atualiza visitas e vitorias ate a raiz.

A formula de selecao combina exploitation, exploration (UCT) e um **bias heuristico fraco**

Um ponto importante e que a heuristica nao substitui o UCT. Ela entra apenas como um prior fraco na selecao dos filhos ja expandidos. A expansao continua a criar filhos a partir de movimentos legais, e o termo de exploracao continua a dar pressao a ramos pouco visitados.

```
In [20]: random.seed(7)
empty_game = PopOut()
empty_game.turn = RED_PLAYER

start = perf_counter()
```

```

mcts_move = MCTS(iterations=1000, rollout_limit=120).search(empty_game)
elapsed = perf_counter() - start

print("Movimentos legais na raiz:", empty_game.available_moves())
print("Movimento escolhido pelo MCTS:", mcts_move)
print(f"Tempo: {elapsed:.3f}s")

```

Movimentos legais na raiz: [(1, 'd'), (2, 'd'), (3, 'd'), (4, 'd'), (5, 'd'), (6, 'd'), (7, 'd')]

Movimento escolhido pelo MCTS: (3, 'd')

Tempo: 6.293s

## Melhorias implementadas sobre o MCTS base

| Melhoria                                          | Função                                                             | Justificacao                                                              |
|---------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------|
| UCT com bias heuristico                           | <code>Node.best_child</code> e <code>MCTS._score_move_prior</code> | Mantem exploracao, mas favorece jogadas taticamente melhores              |
| Prioridade a vitoria imediata                     | <code>MCTS._immediate_winning_moves</code>                         | Evita perder oportunidades obvias nos rollouts                            |
| Bloqueio de vitoria imediata do adversario        | <code>MCTS._blocks_immediate_win</code>                            | Reduz simulacoes que ignoram ameacas de uma jogada                        |
| Penalizacao de respostas vencedoras do adversario | <code>MCTS._opponent_can_win_after</code>                          | Evita jogadas que entregam uma vitoria direta                             |
| Preferencia por colunas centrais                  | <code>MCTS._center_score</code>                                    | Em Connect-4/PopOut, colunas centrais participam em mais linhas possiveis |
| Deteccao de repeticao no rollout                  | <code>MCTS._is_repeated_draw</code>                                | Aproxima a regra de empate por tres repeticoes                            |

## Expansion heuristics

Testamos uma variante de UCT com um prior heuristico. O metodo

`MCTS._score_move_prior` atribui a cada movimento um valor normalizado entre -1 e 1 (multiplicados por uma constante 0.25):

- vitoria imediata do jogador atual;
- bloqueio de uma vitoria imediata do adversario;
- penalizacao de movimentos que permitem resposta vencedora imediata;
- preferencia pequena por colunas centrais, porque participam em mais linhas possiveis de quatro pecas.

Esse `prior_score` e multiplicado por `DEFAULT_HEURISTIC_WEIGHT = 0.25`.

Portanto, mesmo o pior movimento heuristico tem apenas `heuristic_bias = -0.25`, e o melhor tem apenas `heuristic_bias = +0.25`. Ou seja, heuristica é um bias, nao um filtro: nenhum movimento legal é removido.



Desvantagem: a politica pode fazer o algoritmo deixar de verificar opcoes melhores. O contraponto é que esse risco existe se o peso heuristico for alto. Neste projeto o peso é pequeno e o prior é limitado. Além disso, o termo de exploracao do UCT permanece ativo. Por exemplo, com `c = sqrt(2)`, `parent_visits = 100` e `child_visits = 1`:

## Tactical rollout

O rollout tatico aparece em `MCTS._choose_rollout_move`. Durante os primeiros `TACTICAL_ROLLOUT_DEPTH = 8` niveis de simulacao, o algoritmo evita rollouts completamente aleatorios quando ha informacao taticamente obvia:

- se existe uma jogada vencedora imediata, essa jogada tem prioridade;
- se o adversario tem uma vitoria imediata, o rollout tenta escolher uma jogada que bloqueie essa ameaca;
- se nao ha tatica urgente, a escolha continua estocastica, mas com uma leve preferencia por colunas centrais;
- depois da profundidade tatica, o rollout fica mais simples para controlar custo computacional e variancia.

A hipotese é que um prior heuristico fraco e rollouts taticos melhoram a eficiencia da busca em PopOut, principalmente com baixos orcamentos de iteracao, guiando mais simulacoes para movimentos plausiveis sem remover a exploracao de UCT.

## Vantagens e desvantagens

| Aspeto                  | Vantagens                                                                                                                | Desvantagens / riscos                                                                                                               |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Prior heuristico no UCT | Melhora a alocao inicial de simulacoes; reduz desperdicio em jogadas obviamente fracas; ajuda quando ha poucas iteracoes | Pode enviesar a busca se o peso for alto; pode atrasar a descoberta de jogadas contraintuitivas; exige calibracao do hiperparametro |
| Rollout tatico          | Produce simulacoes menos absurdas; captura vitorias e bloqueios de uma jogada; reduz ruido nos resultados                | Aumenta o custo de cada simulacao; continua limitado a taticas locais; pode nao ver planos longos                                   |
| Preferencia central     | Usa uma regularidade conhecida de Connect-4/PopOut; é barata de calcular                                                 | Nem sempre a coluna central é melhor; PopOut tem movimentos <code>pop</code> que podem inverter intuicoes posicionais               |

**TODO: Temos que incluir um grafico de qualidade/tempo com diferentes numeros de iteracoes, por exemplo 50 , 100 , 300 , 500 , 1000 , (...), cuidado que isso deve demorar algum tempo), e uma comparacao entre MCTS base e a MCTS com as melhorias (heuristica e rollout tatico).**

```
In [21]: RUN_MCTS_TIMING = True

if RUN_MCTS_TIMING:
    timing_rows = []
    timing_configs = [
        ("MCTS", lambda iterations: MCTS(iterations=iterations, rollout_l
        (
```

```

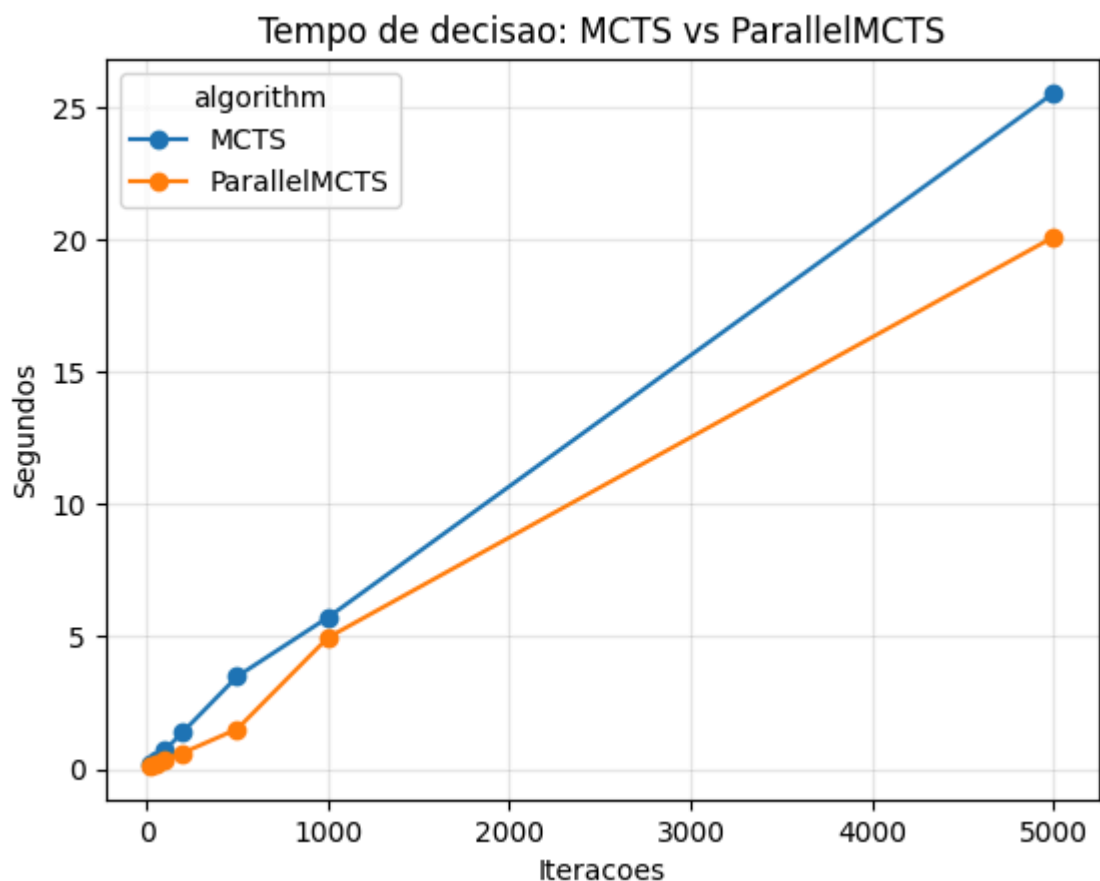
        "ParallelMCTS",
        lambda iterations: ParallelMCTS(
            iterations=iterations,
            rollout_limit=120,
            workers=4,
        ),
    ),
]

for algorithm, build_searcher in timing_configs:
    for iterations in [25, 50, 100, 200, 500, 1000, 5000]:
        game = PopOut()
        game.turn = RED_PLAYER
        start = perf_counter()
        move = build_searcher(iterations).search(game)
        seconds = perf_counter() - start
        timing_rows.append(
            {
                "algorithm": algorithm,
                "iterations": iterations,
                "seconds": seconds,
                "move": str(move),
            }
        )

timing_df = pd.DataFrame(timing_rows)
display(timing_df)
timing_pivot = timing_df.pivot(
    index="iterations",
    columns="algorithm",
    values="seconds",
)
timing_pivot.plot(marker="o")
plt.title("Tempo de decisao: MCTS vs ParallelMCTS")
plt.xlabel("Iteracoes")
plt.ylabel("Segundos")
plt.grid(True, alpha=0.3)
plt.show()
else:
    print("RUN_MCTS_TIMING=False. Mude para True para gerar o grafico de

```

|    | algorithm    | iterations | seconds   | move     |
|----|--------------|------------|-----------|----------|
| 0  | MCTS         | 25         | 0.218789  | (6, 'd') |
| 1  | MCTS         | 50         | 0.315353  | (7, 'd') |
| 2  | MCTS         | 100        | 0.711032  | (2, 'd') |
| 3  | MCTS         | 200        | 1.395968  | (2, 'd') |
| 4  | MCTS         | 500        | 3.499113  | (1, 'd') |
| 5  | MCTS         | 1000       | 5.726428  | (6, 'd') |
| 6  | MCTS         | 5000       | 25.533870 | (4, 'd') |
| 7  | ParallelMCTS | 25         | 0.128870  | (3, 'd') |
| 8  | ParallelMCTS | 50         | 0.198905  | (5, 'd') |
| 9  | ParallelMCTS | 100        | 0.350750  | (3, 'd') |
| 10 | ParallelMCTS | 200        | 0.611692  | (7, 'd') |
| 11 | ParallelMCTS | 500        | 1.515351  | (4, 'd') |
| 12 | ParallelMCTS | 1000       | 4.965335  | (2, 'd') |
| 13 | ParallelMCTS | 5000       | 20.094934 | (5, 'd') |



## 5. Decision Tree ID3

Funcoes principais:

| Ficheiro                          | Funcao                                                      | Descricao                                             |
|-----------------------------------|-------------------------------------------------------------|-------------------------------------------------------|
| <code>metrics.py</code>           | <code>entropy(labels)</code>                                | Mede a impureza da distribuicao d classes             |
| <code>metrics.py</code>           | <code>conditional_entropy(attribute_column, labels)</code>  | Mede a impureza restante apos dividir por um atributo |
| <code>metrics.py</code>           | <code>information_gain(attribute_column, labels)</code>     | Calcula a reducao de entropia causada pelo atributo   |
| <code>metrics.py</code>           | <code>best_attribute(X, labels)</code>                      | Escolhe o atributo com maior ganho de informacao      |
| <code>id3_decision_tree.py</code> | <code>fit(X, labels)</code>                                 | Constroi a arvore recursivamen                        |
| <code>id3_decision_tree.py</code> | <code>predict_one(example)</code>                           | Classifica um exemplo individual                      |
| <code>id3_decision_tree.py</code> | <code>predict(X)</code>                                     | Classifica um DataFrame inteiro                       |
| <code>tree_node.py</code>         | <code>TreeNode.leaf</code> e <code>TreeNode.decision</code> | Representam folhas e nos internos                     |

O ID3 aqui assume atributos categoricos. Por isso, dados numericos como Iris precisam de discretizacao antes do treino.

## 6. Warm-up Iris e discretizacao

O enunciado pede um primeiro dataset Iris com atributos numericos. Como o ID3 trabalha com categorias, o projeto implementa:

- `learn_bins(X_train, columns, n_bins, method)` : aprende limites de intervalos apenas com treino;
- `apply_bins(X, bin_rules)` : aplica os mesmos limites a treino e teste.

Dois metodos estao disponiveis:

- `equal_width` : intervalos com a mesma largura numerica;
- `equal_frequency` : intervalos com aproximadamente o mesmo numero de exemplos.

A célula abaixo executa se existir `data/iris.csv`. Caso contrário, ela apenas indica onde colocar o ficheiro do Moodle. O notebook separado

`iris_id3_notebook.ipynb` contém a exploração feita para este warm-up.

```
In [28]: IRIS_PATH = PROJECT_ROOT / "data" / "iris.csv"

if IRIS_PATH.exists():
    iris = pd.read_csv(IRIS_PATH)
    display(iris.head())

    possible_targets = ["class", "variety", "species"]
    target_col = next(
        (col for col in possible_targets if col in iris.columns),
        None,
    )
    if target_col is None:
        raise ValueError(
            "Nao foi encontrada uma coluna alvo Iris. "
            f"Esperava uma destas colunas: {possible_targets}. "
            f"Colunas encontradas: {list(iris.columns)}"
        )

    feature_cols = [
        col for col in iris.columns if col != target_col and col.lower()
    ]
    print(f"Coluna alvo Iris usada: {target_col}")

    train_df = iris.groupby(target_col, group_keys=False).sample(
        frac=0.8, random_state=42
    )
    test_df = iris.drop(train_df.index)

    X_train = train_df[feature_cols]
    y_train = train_df[target_col]
    X_test = test_df[feature_cols]
    y_test = test_df[target_col]

    rules = learn_bins(X_train, feature_cols, n_bins=3, method="equal_fre
    X_train_discrete = apply_bins(X_train, rules)
    X_test_discrete = apply_bins(X_test, rules)

    iris_tree = ID3DecisionTree().fit(X_train_discrete, y_train)
    predictions = iris_tree.predict(X_test_discrete)
    accuracy = sum(pred == real for pred, real in zip(predictions, y_test
        y_test
    )

    print(f"Iris accuracy: {accuracy:.3f}")
    iris_tree.print_tree()
```

|   | sepal.length | sepal.width | petal.length | petal.width | variety |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Setosa  |

Coluna alvo Iris usada: variety

Iris accuracy: 0.867

Attribute: petal.length

If petal.length == bin\_0:

Attribute: petal.width

If petal.width == bin\_0:

Predict: Setosa

If petal.width == bin\_1:

Predict: Versicolor

If petal.length == bin\_1:

Attribute: petal.width

If petal.width == bin\_0:

Predict: Versicolor

If petal.width == bin\_1:

Predict: Versicolor

If petal.width == bin\_2:

Attribute: sepal.width

If sepal.width == bin\_1:

Predict: Versicolor

If sepal.width == bin\_0:

Predict: Virginica

If petal.length == bin\_2:

Attribute: petal.width

If petal.width == bin\_1:

Attribute: sepal.length

If sepal.length == bin\_1:

Predict: Versicolor

If sepal.length == bin\_2:

Predict: Virginica

If petal.width == bin\_2:

Predict: Virginica

## 7. Dataset PopOut gerado por MCTS

O segundo dataset nao e fornecido pelo enunciado. Ele e construido pelo proprio projeto, guardando pares:

[ (state\_i, move\_i) ]

- **state\_i** : turno atual e 42 celulas do tabuleiro ( **turn** , **cell\_0** , ..., **cell\_41** );
- **move\_i** : melhor movimento sugerido pelo MCTS ( **best\_move** ).

Foram usadas tres fontes possiveis:

| Dataset        | Funcao                               | Ideia                                                                  |
|----------------|--------------------------------------|------------------------------------------------------------------------|
| Random vs MCTS | <code>generate_dataset</code>        | Guarda decisoes do MCTS contra jogadas aleatorias                      |
| MCTS self-play | <code>generate_mcts_self_play</code> | Guarda decisoes quando ambos os jogadores usam MCTS                    |
| MCTS vs DT     | <code>generate_dagger_dataset</code> | Recolhe novos exemplos em estados visitados pela propria Decision Tree |

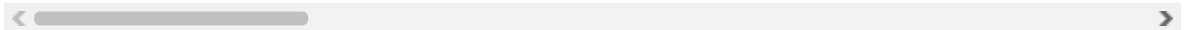
*Nota: todas os arquivos e funcoes estão em src/popout\_decision\_tree/data\_preparation*

A terceira etapa e um estilo DAgger: treina-se uma primeira DT, joga-se contra ela, pede-se novamente ao MCTS a jogada correta e junta-se esse conhecimento ao dataset final.

```
In [31]: popout_df = pd.read_csv(DATASET_PATH)
print("Shape:", popout_df.shape)
display(popout_df.head())
```

Shape: (9887, 44)

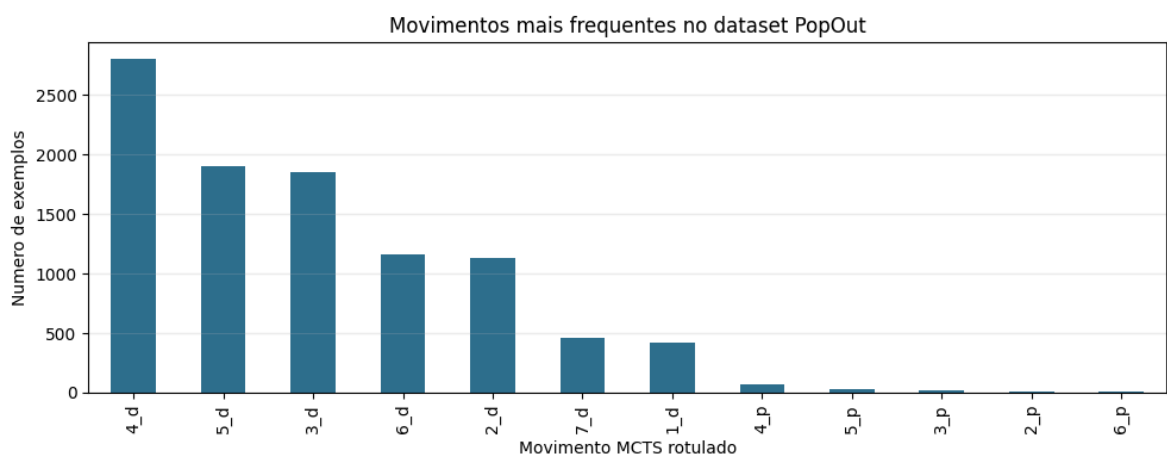
|   | turn | cell_0 | cell_1 | cell_2 | cell_3 | cell_4 | cell_5 | cell_6 | cell_7 | cell_8 | cell_9 | cell_10 |
|---|------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|---------|
| 0 | -1   | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       |
| 1 | -1   | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       |
| 2 | -1   | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       |
| 3 | -1   | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       |
| 4 | 1    | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0       |



```
In [33]: move_counts = popout_df["best_move"].value_counts().sort_values(ascending=False)
display(move_counts.head(15).to_frame("count"))

ax = move_counts.head(12).plot(kind="bar", figsize=(10, 4), color="#2f6f8f")
ax.set_title("Movimentos mais frequentes no dataset PopOut")
ax.set_xlabel("Movimento MCTS rotulado")
ax.set_ylabel("Numero de exemplos")
ax.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()
```

|           | count |
|-----------|-------|
| best_move |       |
| 4_d       | 2802  |
| 5_d       | 1902  |
| 3_d       | 1849  |
| 6_d       | 1165  |
| 2_d       | 1128  |
| 7_d       | 464   |
| 1_d       | 424   |
| 4_p       | 65    |
| 5_p       | 30    |
| 3_p       | 22    |
| 2_p       | 12    |
| 6_p       | 11    |
| 1_p       | 8     |
| 7_p       | 5     |



## 8. Treino da Decision Tree PopOut

A funcao principal e:

```
train_popout_decision_tree(dataset_path, model_path, dot_path)
```

Ela:

1. le o CSV;
2. separa `best_move` como target;
3. treina `ID3DecisionTree().fit(features, target)`;
4. guarda o modelo em `popout_DT_model.pkl`;
5. exporta a arvore para Graphviz DOT.

```
In [36]: RUN_FULL_TRAINING = False # OFF por padrao
```



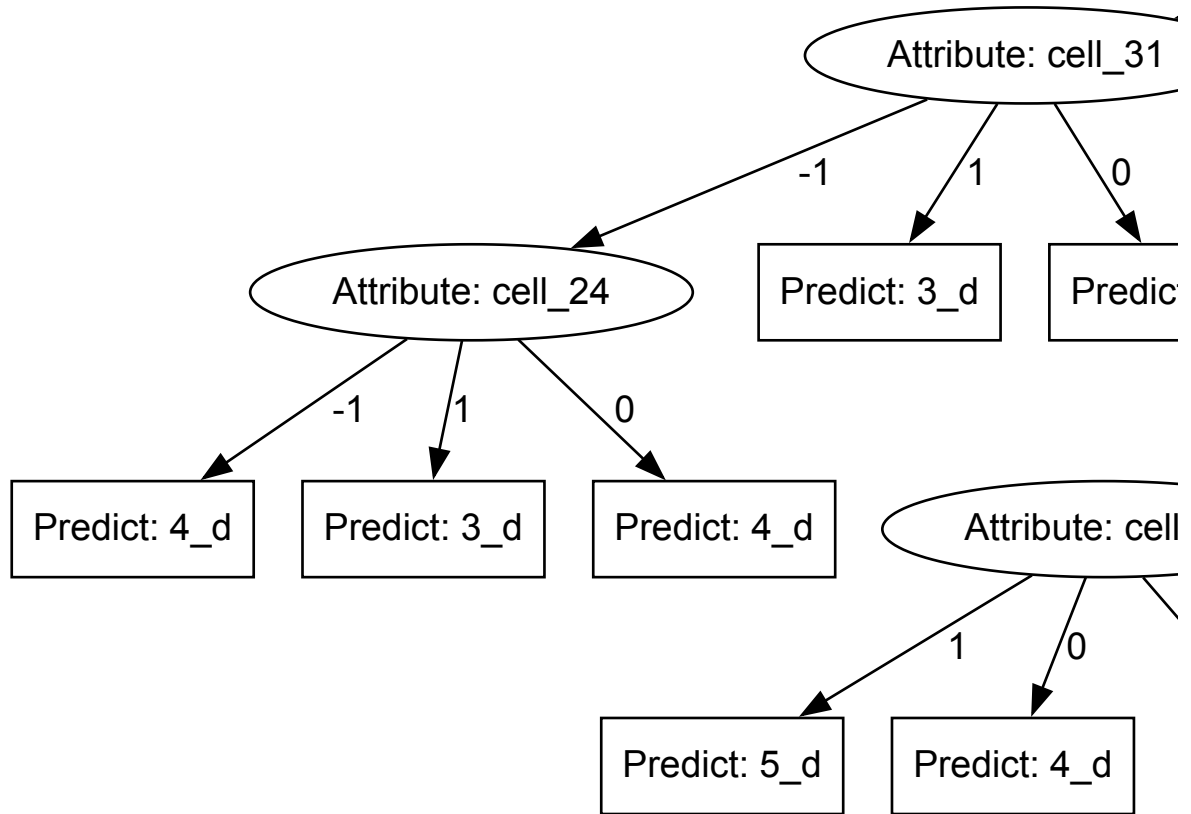
```
if RUN_FULL_TRAINING:
    trained_tree = train_popout_decision_tree(
        dataset_path=DATASET_PATH,
        model_path=MODEL_PATH,
        dot_path=DOT_PATH,
    )
    print("Modelo treinado:", trained_tree)
else:
    print(
        "RUN_FULL_TRAINING=False. Mude para True para treinar novamente "
        "e exportar o modelo."
    )
```

RUN\_FULL\_TRAINING=False. Mude para True para treinar novamente e exportar o modelo.

## 10. Visualizacao da arvore

```
In [38]: display(SVG(filename=str(TREE_SVG_PATH)))
```





## 11. Pipeline de geracao de dados

As células abaixo mostram as chamadas principais para gerar e juntar datasets. Elas estão desligadas por padrão porque MCTS com muitas iterações e muitos jogos pode demorar bastante.

Ordem recomendada:

1. gerar `random_vs_mcts`;
2. gerar `mcts_self_play`;
3. juntar no dataset inicial;
4. treinar uma primeira DT;
5. gerar `mcts_vs_dt` com DAgger;
6. juntar tudo no dataset final;
7. treinar novamente.

```
In [39]: RUN_DATA_GENERATION = False

if RUN_DATA_GENERATION:
    from src.popout_decision_tree.data_preparation.common import (
        FULL_TRAINING_DATASET,
        FULL_TRAINING_INPUTS,
        INITIAL_TRAINING_DATASET,
        INITIAL_TRAINING_INPUTS,
    )
    from src.popout_decision_tree.data_preparation.mcts_vs_dt import (
        generate_dagger_dataset,
    )
    from src.popout_decision_tree.data_preparation.mcts_vs_mcts import (
        generate_mcts_self_play,
    )
    from src.popout_decision_tree.data_preparation.random_vs_mcts import
        generate_dataset,
    )

    generate_dataset(num_games=100, mcts_iterations=1000)
    generate_mcts_self_play(num_games=10, mcts_iterations=1000)
    merge_datasets(
        input_paths=INITIAL_TRAINING_INPUTS, output_path=INITIAL_TRAINING
    )

    train_popout_decision_tree(dataset_path=INITIAL_TRAINING_DATASET)

    generate_dagger_dataset(num_games=20, mcts_iterations=1000)
    merge_datasets(input_paths=FULL_TRAINING_INPUTS, output_path=FULL_TRA
    train_popout_decision_tree(dataset_path=FULL_TRAINING_DATASET)
else:
    print("RUN_DATA_GENERATION=False. Ative apenas quando quiser regenera
```

RUN\_DATA\_GENERATION=False. Ative apenas quando quiser regenerar datasets.

## 12. Demonstracao dos modos de jogo

O ficheiro `src/play_popout.py` implementa os três cenários do enunciado:

| Cenário                   | Comando                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------|
| Humano vs humano          | <code>python -m src.play_popout --red human --blue human</code>                               |
| Humano vs computador MCTS | <code>python -m src.play_popout --red human --blue mcts --mcts-iterations 500</code>          |
| Computador vs computador  | <code>python -m src.play_popout --red mcts --blue dt --mcts-iterations 500 --delay 0.1</code> |

No notebook, não chamamos `main()` diretamente porque o Jupyter adiciona argumentos próprios ao kernel. Em vez disso, podemos importar os jogadores e construir uma partida manualmente quando quisermos uma demonstração interativa.

```
In [41]: from src.play_popout import DecisionTreePlayer, MCTSPlayer, PopOutMatch

match = PopOutMatch(
    red_player=MCTSPlayer(iterations=300, name="Red MCTS"),
    blue_player=DecisionTreePlayer(model_path=str(MODEL_PATH), name="Blue",
    delay=0.1,
)
match.run()

print("Classes importadas: RandomPlayer, MCTSPlayer, DecisionTreePlayer,
```

=== POP OUT MATCH ===

RED: Red MCTS

BLUE: Blue Decision Tree

=====

```

|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|

```

RED's turn!

↓ ↓ ↓ ↓ ↓ ↓ ↓

```

|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|

```

Red MCTS is choosing a move...

RED (Red MCTS) played: column 5, drop

```

|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|

```

BLUE's turn!

↓ ↓ ↓ ↓ ↓ ↓ ↓

```

|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|

```

Blue Decision Tree is choosing a move...

BLUE (Blue Decision Tree) played: column 5, drop

```

|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|

```

RED's turn!

↓ ↓ ↓ ↓ ↓ ↓ ↓

```

|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|
|●|●|●|●|●|●|●|

```

↓

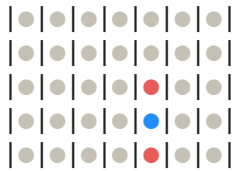
Red MCTS is choosing a move...

RED (Red MCTS) played: column 5, drop

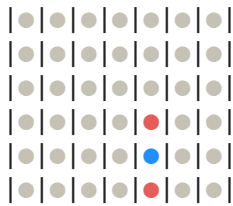
```

|●|●|●|●|●|●|●|

```

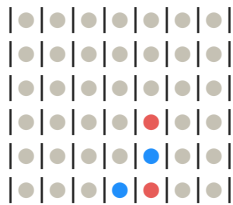


BLUE's turn!

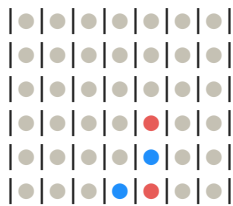


Blue Decision Tree is choosing a move...

BLUE (Blue Decision Tree) played: column 4, drop

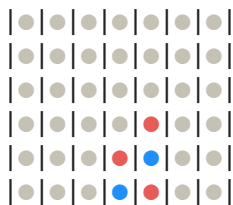


RED's turn!

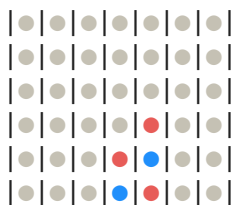


Red MCTS is choosing a move...

RED (Red MCTS) played: column 4, drop

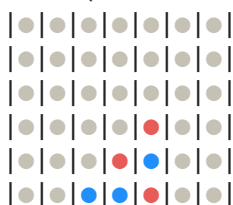


BLUE's turn!

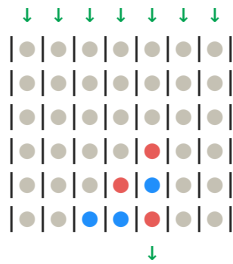


Blue Decision Tree is choosing a move...

BLUE (Blue Decision Tree) played: column 3, drop

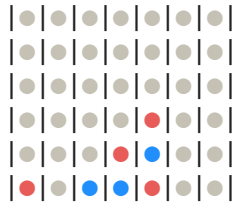


RED's turn!

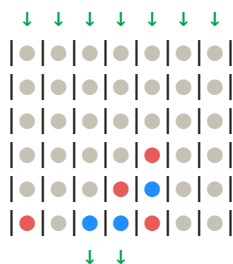


Red MCTS is choosing a move...

RED (Red MCTS) played: column 1, drop

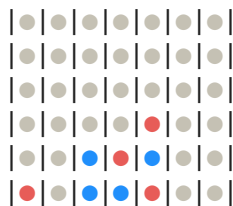


BLUE's turn!

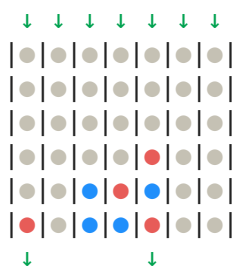


Blue Decision Tree is choosing a move...

BLUE (Blue Decision Tree) played: column 3, drop

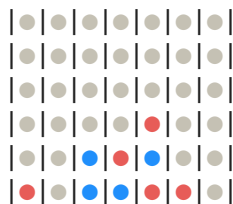


RED's turn!

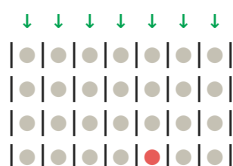


Red MCTS is choosing a move...

RED (Red MCTS) played: column 6, drop



BLUE's turn!







Blue Decision Tree is choosing a move...

**BLUE** (Blue Decision Tree) played: column 4, pop

**RED's turn!**




Red MCTS is choosing a move...

**RED** (Red MCTS) played: column 7, drop

**RED WINS!**

Classes importadas: RandomPlayer, MCTSPlayer, DecisionTreePlayer, PopOutMa  
tch

## 13. Discussao dos resultados

Pontos fortes da solucao:

- o motor PopOut separa regras do jogo, visualizacao terminal e agentes;
- o MCTS e generico e pode ser reutilizado noutros jogos que cumpram o contrato;
- a Decision Tree ID3 e implementada manualmente com entropia e ganho de informacao;
- o dataset PopOut e produzido por uma politica forte, MCTS, em vez de labels manuais;
- o pipeline DAGger reduz o problema de a Decision Tree encontrar estados que nao viu no primeiro treino;
- a interface final suporta os tres modos pedidos no enunciado;
- as heurísticas de expansao e o rollout tatico sao apresentados como hipoteses experimentais, nao como garantias teoricas.

Limitacoes e melhorias propostas:

- comparar quantitativamente `random`, `MCTS`, `ParallelMCTS` e `DecisionTree` em dezenas de partidas;
- reportar tempo medio por jogada para diferentes iteracoes MCTS;

- podar ou limitar a árvore para reduzir tamanho e melhorar interpretabilidade;
- testar o peso heurístico `DEFAULT_HEURISTIC_WEIGHT` com valores diferentes, por exemplo `0.0`, `0.1`, `0.25` e `0.5`;
- isolar o impacto do rollout tático comparando rollouts aleatórios contra rollouts com vitórias/bloqueios imediatos.